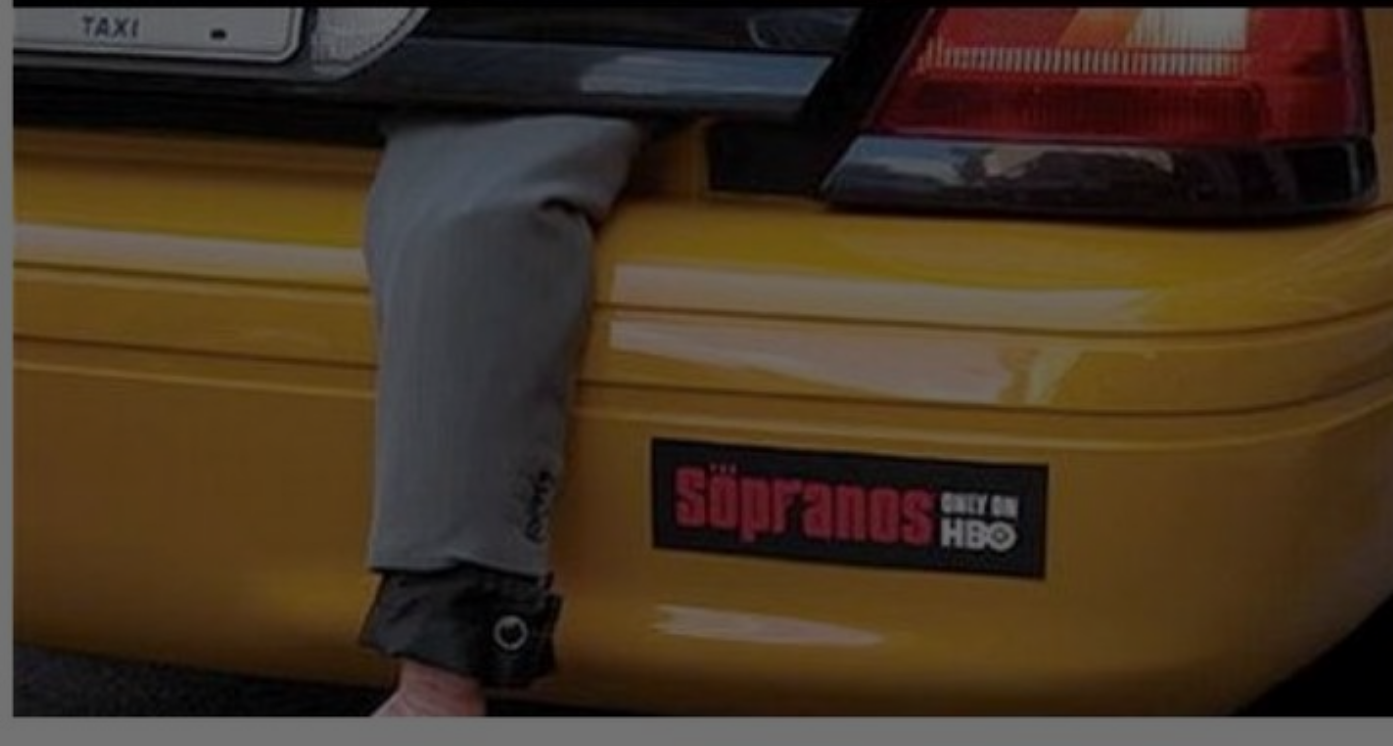
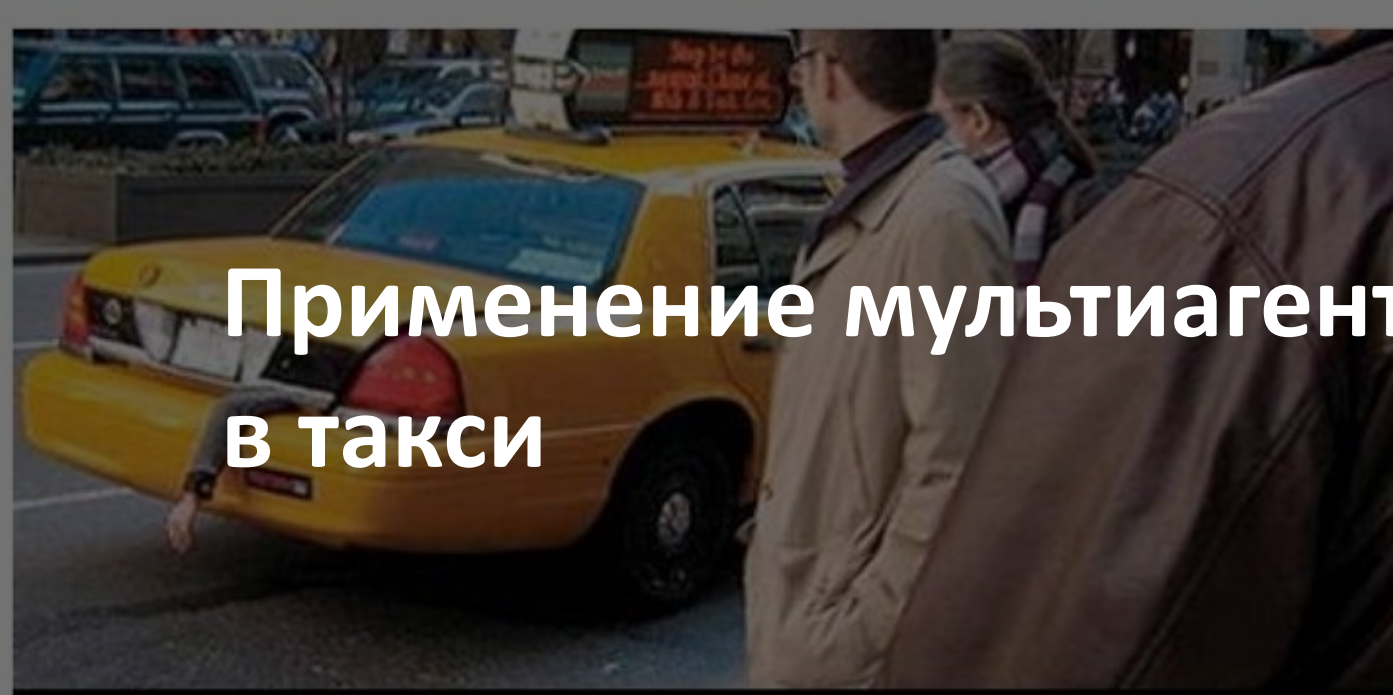
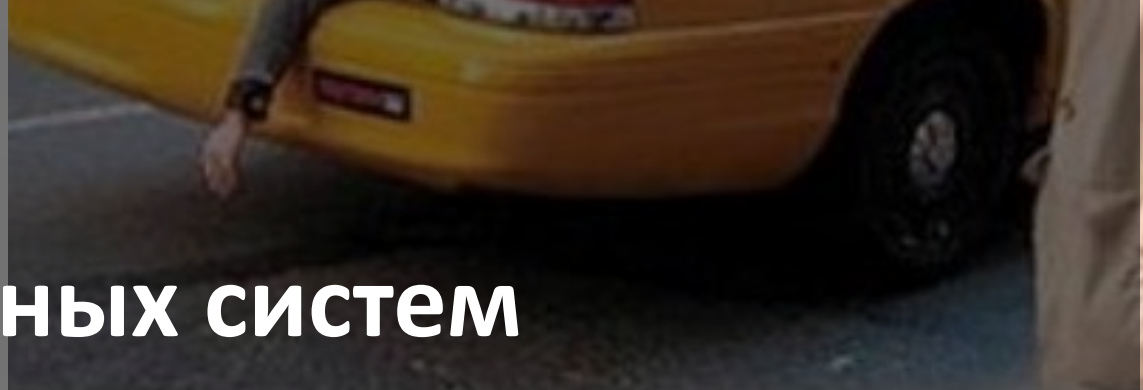


Применение мультиагентных систем в такси



Транспорт = сеть + неопределённость + поток событий

Почему сети растут

- Крупные парки лучше обслуживают крупных клиентов (качество/выгоды)
- Конкуренция + рост цен (например, топлива) → укрупнение компаний
- ERP-системы зрелые, но real-time управление остаётся проблемой

Почему «классика» не успевает

- Слишком много событий и ограничений одновременно
- Данные есть (GPS, карты, трафик), но их трудно учесть в оптимизации
- На практике всё держится на опытных диспетчерах и эвристиках
- Потенциал эффекта: 3–15% эффективности при правильном real-time управлении

масштабы real-time диспетчеризации (Москва)

20000+

Водителей

100 000+

заказов в день

5000/ч

пики потока заказов

~4000

водителей онлайн одновременно

≤2 мин

Время подбора водителя

Что усложняет задачу:

- Заказы бывают срочные и заранее забронированные; важность клиента: 0–100
- Типы услуг: минивэн, VIP и т.п.; спецтребования: животные, детское кресло, оплата картой
- Водители независимые: гибкое начало/конец смены; конкурируют за заказы
- Неопределённость: пробки, очереди в аэропортах/на вокзалах, no-show, поломки

Event-driven

- Поступление новых заказов (срочных и заранее)
- Изменения/отмена заказа
- Смена профиля водителя/машины (тип, оснащение)
- Смена статуса водителя: свободен, занят, перерыв, «свободен через 5/10 мин», «еду домой»
- Смена координат (GPS), дорожные инциденты, пробки
- Неявка клиента, поломка автомобиля

Ограничения и «исключения»

- Совместимость: класс авто, оборудование, требования клиента
- Приоритеты: VIP и важность (0–100)
- Сервис: подача ≤ 15 минут в центре
- Справедливость: кто дольше без заказов
- Очереди (аэропорт/вокзал): порядок ожидания
- «По пути домой»: назначать поездки в нужном направлении
- Правила могут изменяться «на лету»

Формально это «задача назначения», но real-time делает её тяжёлой

Проблема 1: вычислительная сложность

Типичные оценки сложности алгоритмов для назначения: $O(n^4)$

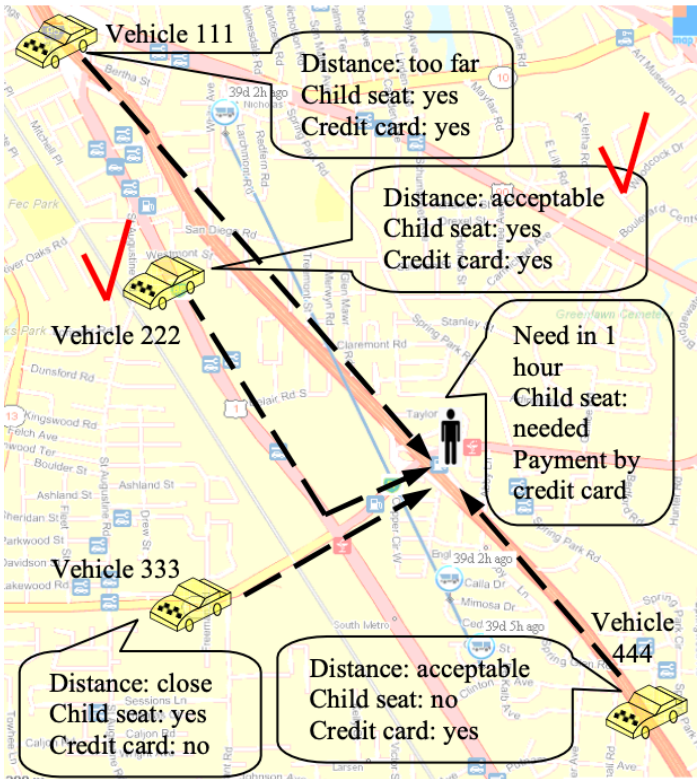
- Статус и координаты меняются каждые ~30 секунд
- Частые события → требуется постоянное перепланирование
- Полный пересчёт становится неприемлемым по времени

Проблема 2: правила «условные» и меняются

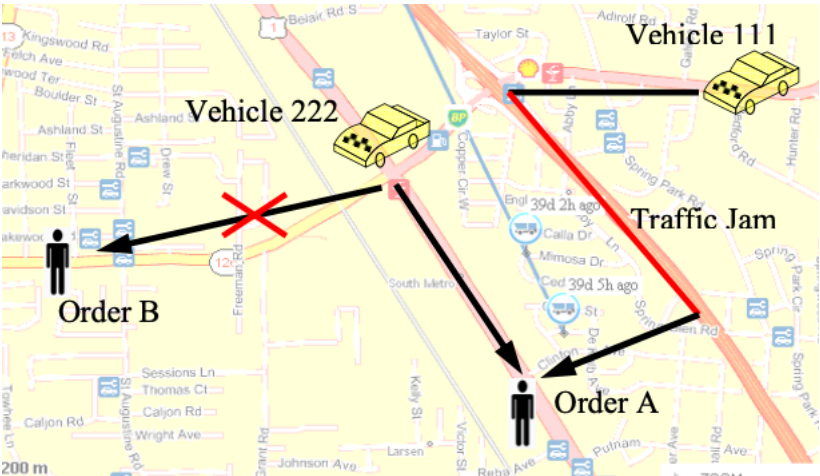
- Базово выбираем по расстоянию, но при равенстве — по «кто дольше без заказа»
- Очереди (аэропорт) добавляют свои правила
- VIP может «перебить» текущее назначение
- Набор критериев зависит от текущей ситуации

Что делают обычно:

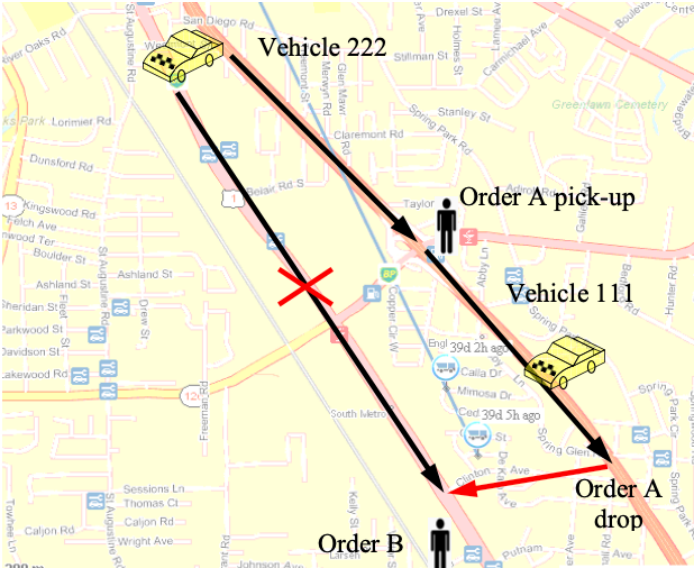
- Метаэвристики: GA, ACO, tabu, adaptive memory



Доступный пул



Оценка происходящего



Выбор водителя

MAS помогает, когда нужно быстро согласовать много локальных решений

Пример 1: заказ с набором требований

- «Через 1 час + детское кресло + оплата картой»
- Агент заказа опрашивает кандидатов
- Сравнивает предложения и выбирает лучшее
- Можно добавить бонусы для справедливости

Пример 2: реакция на событие

- Случилась пробка → риск сорвать подачу
- Водитель запрашивает «страховочную сеть»
- Если рядом освободился другой водитель — берёт заказ
- Переговоры идут внутри системы, без радио-хаоса

Пример 3: VIP и перераспределение

- VIP-заказ имеет высокий приоритет
- Даже если «идеальная» машина уже назначена
- VIP получает ресурс, а предыдущий заказ переназначается
- Так достигается нужный уровень сервиса

Ключ: агенты позволяют распределённо принять решение и договориться о переназначениях.

Идея №1

Локальная адаптация

Когда случается событие (новый заказ, пробка, смена статуса водителя), система не перестраивает всё расписание. Она модифицирует только те фрагменты, которые реально затронуты.

Что это даёт:

- Скорость: меньше вычислений на каждое событие
- Стабильность: не «ломаем» весь план при локальном возмущении
- Гибкость: можно быстро внедрять новые правила/критерии

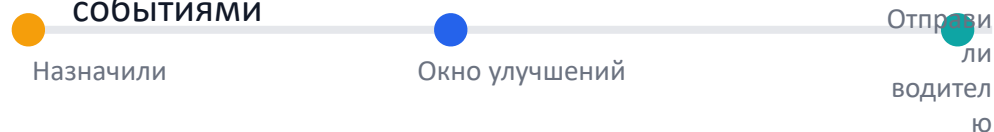
Идея №2

Задержка подтверждения назначения

Между моментом «назначили машину» и моментом «отправили инструкции водителю» есть окно времени. В это окно назначение считается предварительным и может изменяться, если это улучшает KPI.

Следствие: «волна» переназначений

- Новый заказ может конфликтовать со старым назначением
- Система допускает цепочку переназначений ($A \rightarrow B \rightarrow C \dots$)
- Длина цепочки ограничена доступным временем до подачи
- Расписание строится и постоянно уточняется между событиями



Ключевая идея: нет единого «центра», который считает всё заново — решения возникают из переговоров агентов и локальных переназначений.

Роли агентов

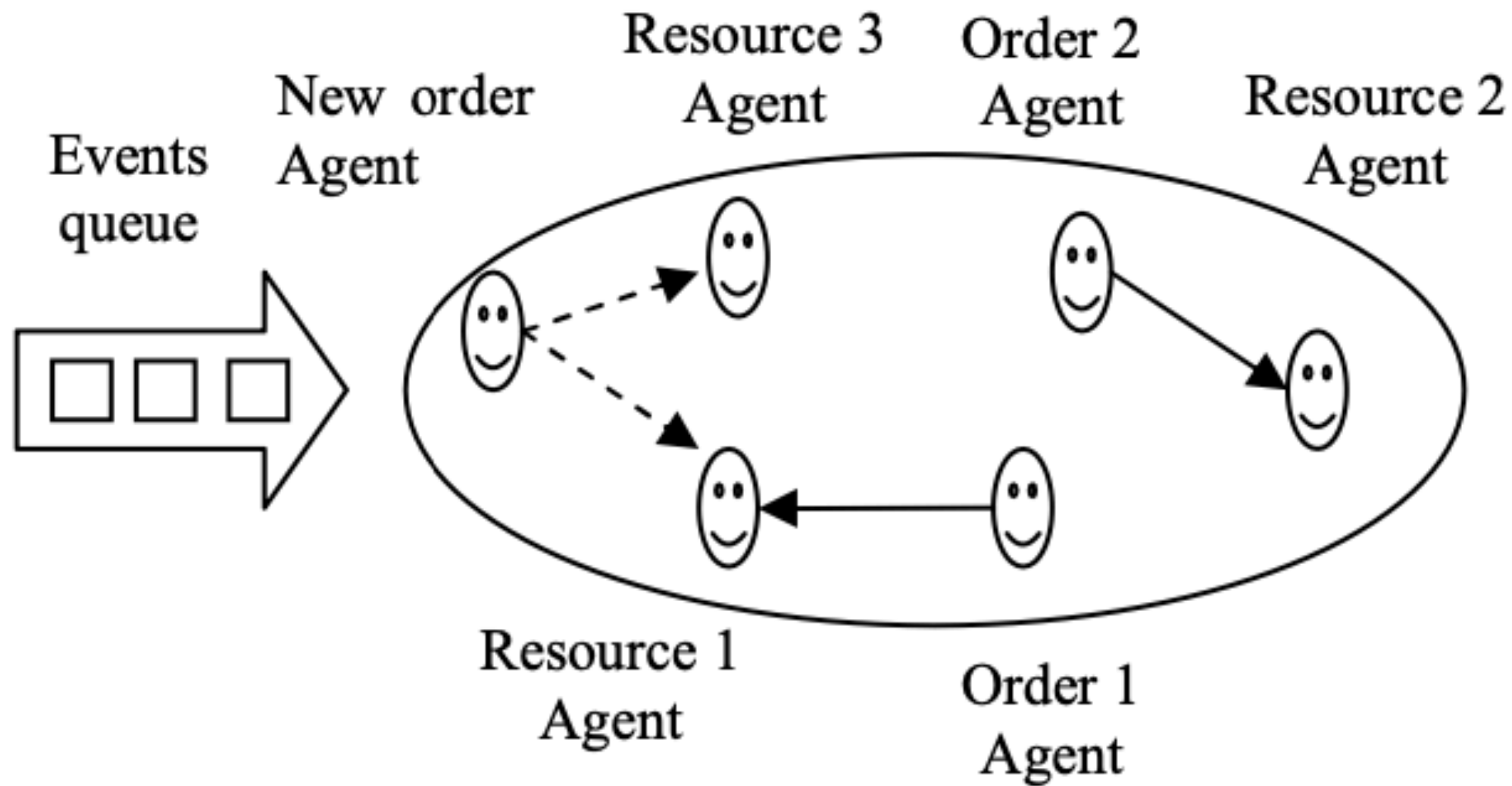
- Агент заказа (Order Agent) создаётся на каждый новый заказ . Он активно: формирует список кандидатов и инициирует переговоры
- Агент ресурса (Driver/Resource Agent) представляет водителя/машину
- В первой версии он реактивен: отвечает ставкой и исполняет выбранное решение
- Доп. агенты/модули: процессор внешних событий, менеджер перегруженных зон, менеджер распределения заказов

Отличие от «объекта»: автономность (цель) + коммуникация (сообщения)

Цикл работы (event-driven)

- 1) Система собирает события в очередь (заказы, отмены, пробки, статусы)
- 2) Диспетчер событий берёт событие и «передаёт ход» агентам
- 3) Агент заказа запрашивает ставки у агент(ов) ресурсов
- 4) Сравнение ставок по актуальным критериям → выбор лучшего
- 5) При конфликте запускаются переназначения (цепочки)
- 6) Когда очередь пуста — система «спит» до нового события

события → переговоры → (возможно) переназначения → обновлённое расписание.



Два шага: (1) сузить кандидатов, (2) ранжировать по многим критериям

Шаг 1. Предварительное сопоставление (pre-matching)

- Отсекаем явно неподходящих водителей/машины
- Фильтры: тип сервиса, оборудование, спецтребования
- Ограничиваем радиус/«видимость» по расстоянию
- Результат: резко меньше пар «заказ–водитель» для оценки

Шаг 2. «Микроэкономика»: многокритериальная оценка

Оценка пары заказ–водитель = сумма критериев × веса (хорошие варианты кэшируются)

- Расстояние до заказа и прогноз задержки
- Предпочтения/справедливость: кто дольше простаивает
- Опыт водителя
- Близость к перегруженной зоне (переток ресурсов)
- Важность и приоритет заказа (VIP)

Дальше: переговоры и ставки (bids)

- Новый агент заказа запрашивает «стоимость выполнения» у выбранных ресурсов
- Если ресурс занят, его ставка включает стоимость переноса текущего заказа
- Сравниваем ставки и выбираем лучшее под текущий критерий
- Если новое решение улучшает KPI — применяем и продолжаем улучшать до истечения времени

Идея: разрешаем цепочки переназначений, но ограничиваем глубину поиска

b — глубина поиска, k — число переносимых заказов (в тексте: $k = 1$, $b < 5$).

Search depth	Scheduling time	Evaluation mark
1	—	198.98
2	—	200.28
3	—	198.98
4	0.1 sec	198.87
5	1.3 sec	198.87
6	14.87 sec	198.87
7	532.87 sec	198.87

Выводы из Таблицы

- Качество (оценка) стабилизируется уже на $b = 4$
- Увеличение b почти не улучшает результат, но время взлетает
- $b=4$ даёт оптимум в 99% случаев
- Практика: средняя глубина ≈ 1.5 , растёт в пик

Интерпретация для real-time: «ограничиваем поиск, иначе оптимум придёт завтра, когда заказ уже отменили».

- При одновременном распределении 100 входящих заказов $b=4$ занимает ≈ 0.1 секунды
- Значит, есть время на несколько итераций улучшения между событиями

Как обрабатывается новый заказ (по шагам)

Часть А: поиск кандидатов

- 1) Заказ попадает в очередь событий
- 2) Проверка возможности планирования
- 3) Заказ получает агента (Order Agent)
- 4) Pre-matching: оставляем только подходящих водителей
- 5) 5) Оцениваем пары «водитель–заказ» по критериям и весам
- 6) Агент заказа запрашивает ставки (стоимость/время) у кандидатов

Часть В: улучшения и фиксация

- 7) Если новое решение лучше текущего — применяем
- 8) Продолжаем опрос для кандидатов, чья исходная оценка лучше текущей
- 9) Если в цикле нет изменений — событие обработано
- 10) Для заказа задаётся динамическое «время принятия решения»
- 11) Время истекло → отправляем инструкции водителю и фиксируем заказ

В реальном городе алгоритм обязан учитывать поведение людей и «аномалии»

1) Перегруженные районы (demand spike)

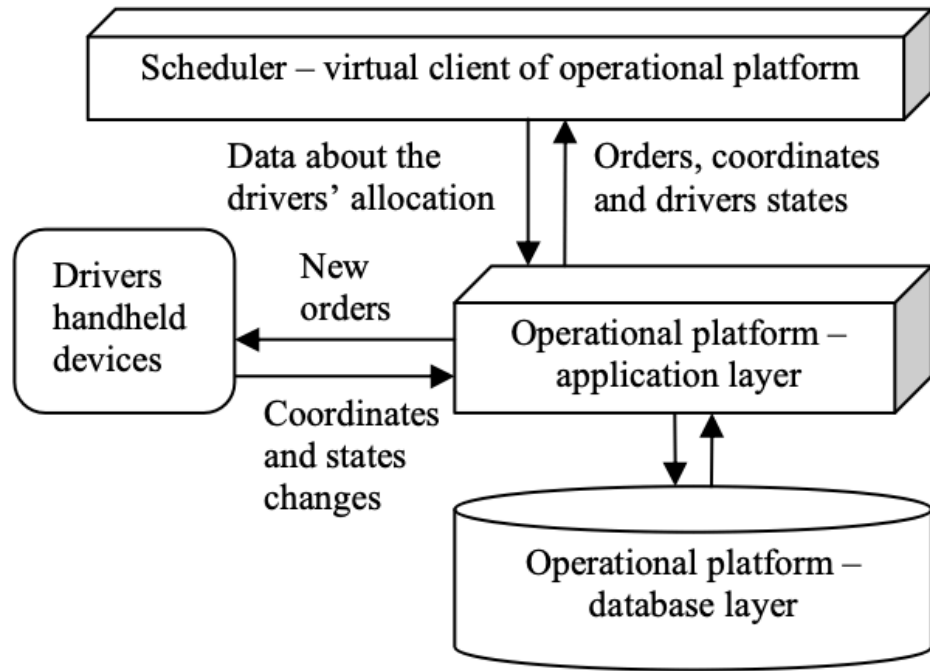
- Поток заказов «взрывается», зона остаётся без водителей на часы
- В нормальном режиме водителей, едущих в центр, «перехватывают» ближние заказы
- Решение: временно менять критерии распределения и ограничения видимости
- Дополнительно: флаг на платформе — ограничить приём срочных заказов низкого приоритета в критической зоне

2) Прогноз спроса на 30 минут и «подсказки» водителям

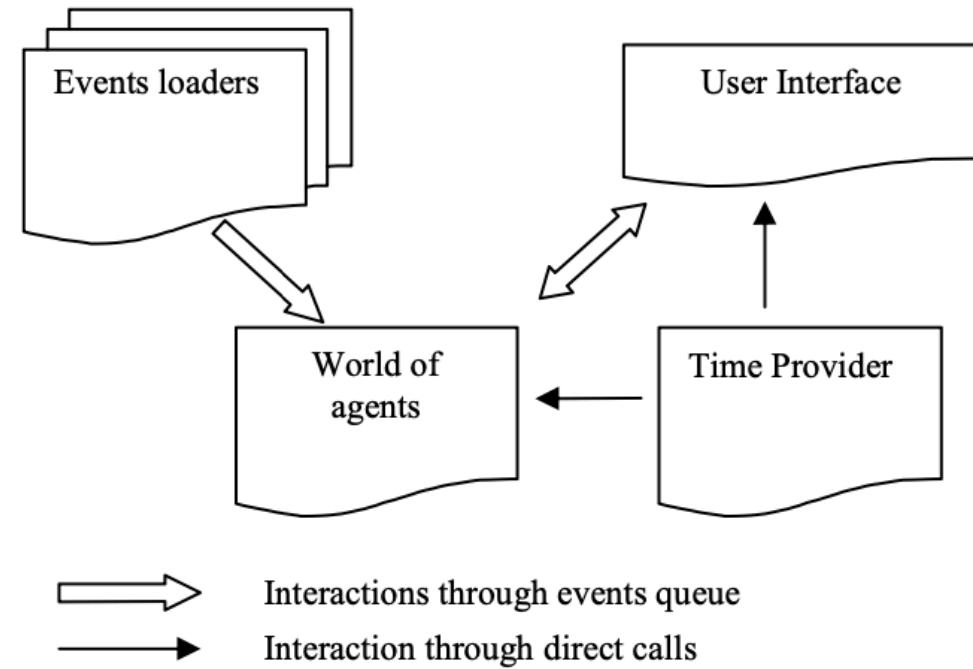
- Без управления предпочтениями возможен перекосяк: избыток водителей в одном районе и дефицит в другом
- По истории + текущему потоку строится краткосрочный прогноз
- Свободным водителям отправляются рекомендации остаться/переместиться
- Эффект: меньше время отклика и меньше пробег «впустую»

3) Античит: попытки обмана статусов

- «Я в очереди аэропорта», хотя водитель за десятки миль
- «Свободен через 10 минут», хотя только начал текущий заказ
- Многократное «еду домой» ради выгодных направлений
- Решение: агенты мониторят расписание и при необходимости игнорируют запросы



Взаимодействие планировщика с платформой



Архитектура планировщика

Планировщик работает как «виртуальный клиент» операционной платформы: получает данные, принимает решения и возвращает назначения.

1) Операционная платформа (ERP) - Вход: новые заказы из колл-центра и веба - Вход: координаты и статусы водителей (GPS, кнопки статуса) - Выход: данные о назначениях и изменениях Слой: БД + уровень приложения (через сервер приложений)	2) Планировщик (виртуальный клиент) - Очередь событий: события собираются и обрабатываются по одному «Мир агентов»: агенты заказов и ресурсов ведут переговоры - Модули и многопоточность; конфигурация меняется по необходимости Работает 24/7 и автономно распределяет ресурсы	3) Устройства водителей + интерфейс - Получают задания (инструкции по подаче/маршруту) - Отдают статусы и подтверждения, обновляют координаты - UI для операторов: список заказов и назначения Карты: маршрут и текущее положение автомобиля
---	---	---

Инженерные инструменты для тестирования и анализа:

- загрузчики XML-сценариев и исторических данных;
- подмена «поставщика времени» (реальное/модельное время, пошаговый прогон);
- воспроизведение ситуаций для оценки качества расписаний.